TECHNICAL CASE STUDY

CredVault

A Premium Fintech Microservices Platform

Angular 19 | .NET 8 / 9 | Microservices | RabbitMQ | Event-Driven | Ocelot Gateway | NgRx | SQL Server | Docker | CQRS + MediatR

Prepared by **Antigravity** · Senior Software Architect · 2025

CONTENTS

Table of Contents

CredVault — Full Technical Case Study · 15 Sections

SECTION 01

Executive Summary

CredVault is a state-of-the-art fintech application providing a unified platform for managing credit cards, utility bills, and digital payments. Built on a modern microservices architecture with a high-performance Angular 19 frontend, the system delivers a premium, high-security experience for modern financial users.

The project achieves full separation of concerns through five core microservices — **Identity, Card, Billing, Payment, and Notification** — orchestrated via an Ocelot API Gateway. It leverages Event-Driven Architecture (EDA) with RabbitMQ to ensure loose coupling and high reliability, enabling real-time notifications and automated reward points calculation upon payment completion.

SECTION 02

Problem Statement

Managing multiple credit cards, tracking bill due dates, and ensuring secure payment processing often results in a fragmented user experience. Before CredVault, users had to jump between different banking apps, manually track rewards, and frequently missed payment deadlines due to a lack of centralized notifications.

Fragmented Experience

No single source of truth for cards, bills, and payments across multiple banking providers.

Missed Deadlines

No automated notification system meant users frequently missed bill due dates.

No Rewards Visibility

Reward points were siloed and difficult to track across multiple card providers.

Security Gaps

Traditional apps lacked MFA, exposing users to credential theft and session hijacking.

SECTION 03

Goals & Objectives

Functional Goals

- **Centralized Identity Management:** Secure user onboarding, authentication, and session management with MFA.
- **Card Portfolio Management:** Complete lifecycle management of credit and debit cards.
- **Automated Billing:** Real-time tracking of utility and credit card bills with automated status updates.
- **Transaction Processing:** Robust payment engine supporting UPI, Bank Transfer, and Card payments.
- **Communication Hub:** Reliable delivery of OTPs and transaction receipts via email notifications.

Non-Functional Goals

- **Scalability:** Microservices allow independent scaling of high-load services like Payments.
- **Security:** JWT-based auth, Refresh Tokens, BCrypt hashing, and Multi-Factor Authentication.
- **Reliability:** Asynchronous event processing via RabbitMQ prevents single-service failures from blocking user flows.
- **User Experience:** Premium "glassmorphic" UI with smooth transitions and 3D card visualizations via Three.js.

SECTION 04

Scope

In-Scope

- Full-stack microservices (.NET + Angular)
- API Gateway integration and routing
- Event-driven inter-service communication
- Security infrastructure (Auth, MFA, Tokens)
- Database per service with EF Core
- Responsive frontend with NgRx state

Out-of-Scope

- Live banking API integration (simulated)
- Native mobile apps (iOS / Android)
- Blockchain-based immutable ledger

SECTION 05

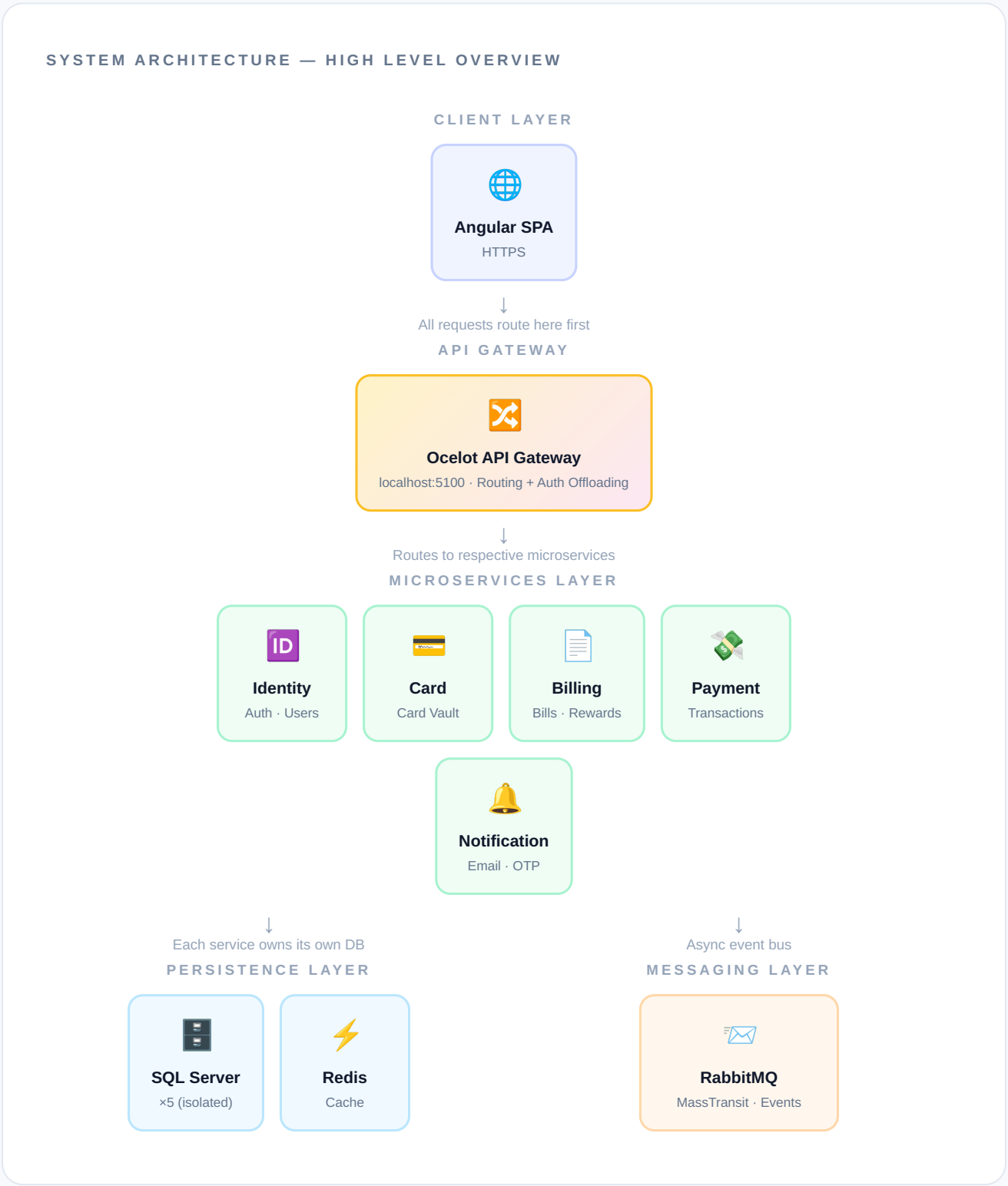
Tech Stack

Layer	Technologies
Frontend	Angular 19, Standalone Components, Signals, NgRx, RxJS, Lucide Angular, Three.js, ECharts
Backend	.NET 8/9 Core, ASP.NET Web API, MediatR (CQRS), Ocelot (API Gateway)
Database	SQL Server 2022, Entity Framework Core, Redis (Caching)
Messaging	RabbitMQ, MassTransit (Message Broker)
Security	JWT, Refresh Tokens, BCrypt Password Hashing, MFA via OTP
DevOps	Docker, Docker Compose

SECTION 06

System Architecture

The system follows a **Microservices Architecture** with **Clean Architecture** inside each service. All traffic flows through a single Ocelot API Gateway, which routes requests downstream and offloads authentication concerns.



Architectural Highlights

- **API Gateway (Ocelot):** Single entry point for all client traffic. Handles routing and auth offloading so each microservice doesn't implement its own auth checks.
- **Database per Service:** Each microservice manages its own isolated schema, enabling independent deployments and zero cross-service data coupling.

- **MassTransit + RabbitMQ:** Asynchronous event bus. When a user registers, Identity publishes an event; Notification consumes it and sends a welcome email — no blocking of the registration response.

SECTION 07

Module & Feature Breakdown

🆔 Identity Service

Responsibility: Source of truth for user data and security. Uses MediatR for CQRS. Login generates a short-lived JWT plus a Refresh Token persisted in SQL Server. MFA generates 6-digit OTPs with a 5-minute expiry, consumed and invalidated after verification.

💳 Card Service

Responsibility: Manages the user's card vault. Allows linking and unlinking cards. Card numbers are masked and encrypted as a PCI-compliance foundation, associated with a UserId and stored in the isolated CardDB.

📄 Billing Service

Responsibility: Tracks financial obligations and loyalty. Listens for PaymentCompletedEvent. On receipt, marks the corresponding bill as Paid and calculates reward points at a configurable ratio (₹100 = 1 Point).

💸 Payment Service

Responsibility: Transaction processing engine. Implements a Saga-lite pattern — creates a record in Processing state, simulates gateway communication, then transitions to Completed or Failed and publishes the matching event.

🔔 Notification Service

Responsibility: Delivery of all outbound communications. A pure consumer service using MassTransit to react to system events — OTPRequestedEvent, UserRegisteredEvent, PaymentCompletedEvent — and dispatches styled HTML emails via SMTP.

SECTION 08

Sequence Diagrams — Key Flows

A. User Authentication & MFA Flow

AUTHENTICATION + MFA SEQUENCE

User	Frontend	Gateway	Identity Svc	RabbitMQ	Notif. Svc

1. User enters email and password → Frontend sends `POST /api/auth/login` to the Gateway.
2. Gateway routes to Identity Service → validates credentials via BCrypt hash comparison against the DB.
3. Identity Service detects MFA is enabled → publishes `OTPRequestedEvent` to RabbitMQ (non-blocking).
4. Identity Service immediately returns `200 MFA_REQUIRED` to Frontend — the auth API stays fast and responsive.
5. Notification Service consumes the event from RabbitMQ → sends styled HTML OTP email to the user's inbox.
6. User reads the 6-digit OTP from email → enters it in the MFA challenge screen on the Frontend.
7. Frontend sends `POST /api/auth/verify-otp` → Identity Service validates the OTP against the stored hash (5-min expiry check).
8. Identity Service returns **JWT Access Token (15 min)** + **Refresh Token (7 days, HTTP-only cookie)** → user is fully authenticated.

Key Design Decision: By publishing an OTPRequestedEvent rather than sending the email synchronously inside the login handler, the Identity Service stays decoupled and fast. If the email provider is temporarily unavailable, RabbitMQ holds the event and retries — the auth service continues functioning.

B. Payment Processing & Billing Settlement Flow

PAYMENT + EVENT FAN-OUT SEQUENCE

User	Frontend	Payment Svc	RabbitMQ	Billing Svc	Notif. Svc

1. User clicks "Pay Bill" → Frontend sends `POST /api/payments/pay` with bill ID, amount, and payment method.
2. Payment Service creates a transaction record with status `PROCESSING` (Saga-lite: ensures no transaction is lost if the system crashes mid-flow).
3. Payment Service simulates external gateway communication and evaluates the result asynchronously.

🟡 **SUCCESS PATH**

- → Payment Service marks record `COMPLETED` in the database.
- → Publishes `PaymentCompletedEvent` to RabbitMQ with amount and billId.
- → Returns `200 OK` with transaction reference to the Frontend immediately.

🔴 **FAILURE PATH**

- → Payment Service marks record `FAILED` (compensation step executed).
- → Publishes `PaymentFailedEvent` → Notification Service alerts user via email.
- → Returns `400 Error` with failure reason to the Frontend.

🟢 **PARALLEL — ON SUCCESS (FAN-OUT)**

- → RabbitMQ → **Billing Service:** Mark bill as PAID, calculate and credit reward points.
- → RabbitMQ → **Notification Service:** Send payment receipt email to user.

Eventual Consistency: Even if Billing is temporarily down when the PaymentCompletedEvent fires, RabbitMQ holds the message. Once Billing recovers, it processes the event and marks the bill paid — no manual intervention required. This is the core benefit of the Saga pattern applied here.

SECTION 09

Frontend Architecture & Flow

The frontend is an **Angular 19 Standalone Application** using a fully reactive architecture — NgRx for global cross-component state and Angular Signals for fine-grained local reactivity.

NgRx Global State Domains

Store Domain	What It Manages	Updated When
Auth Store	Logged-in user, JWT, user profile	Login / Logout / Token refresh
Cards Store	User's linked card list (cached)	Card added, removed, or fetched
Payments Store	Transaction status and history	Payment submitted / completed / failed

User Action → UI Update Lifecycle

1. **User Action**

   User clicks "View Details" on a card in the dashboard.

2. **Action Dispatch**

   Component dispatches a `LoadCardDetails` NgRx action to the central store.

3. **Effect Trigger**

   An NgRx Effect intercepts the action and calls the Angular `CardService`.

4. **HTTP Request**

   The Angular service fires `GET /api/cards/:id`. The `AuthInterceptor` auto-attaches the JWT header before the request leaves the browser.

5. **Store Update**

   On success, the Effect dispatches `LoadCardDetailsSuccess` with the payload, updating the NgRx store slice.

6. **Reactive Re-render**

   The UI, subscribed via `async` pipe or Angular Signal, automatically re-renders with new data — zero manual change detection needed.

SECTION 10

Database Design

CredVault uses a relational schema across **five isolated SQL Server databases** — one per microservice — managed via Entity Framework Core.

Entity	Database	Key Fields	Notes
Users	IdentityDB	Id, Email, PasswordHash, IsMfaEnabled	Source of truth for identity
RefreshTokens	IdentityDB	Token, UserId, ExpiresAt, IsRevoked	Supports secure rotation
Cards	CardDB	Id, UserId, CardNumber (Masked), Expiry, Provider	PAN encrypted at rest
Bills	BillingDB	Id, UserId, Category, Amount, DueDate, Status	Status: Unpaid / Paid
Rewards	BillingDB	UserId, PointsEarned, TransactionRef	Ratio: ₹100 = 1 Point
Payments	PaymentDB	Id, UserId, BillId, Amount, Status, TransactionRef	Status: Processing / Completed / Failed

Data Transformation: On payment completion, the `Amount` field maps to a `PointsEarned` calculation inside the Billing Service's event handler using a configurable ratio — keeping the Payment Service clean and focused solely on transaction concerns.

SECTION 11

API Design

All APIs follow RESTful principles and are exposed through the Ocelot Gateway at `http://localhost:5100`.

Endpoint	Method	Purpose	Service
`/api/auth/login`	POST	Authenticate user, initiate MFA	Identity
`/api/auth/verify-otp`	POST	Validate OTP, return JWT + Refresh Token	Identity
`/api/auth/refresh`	POST	Rotate expired access token	Identity
`/api/cards`	GET	List user's linked cards	Card
`/api/cards`	POST	Link a new card to the vault	Card
`/api/bills`	GET	Retrieve all bills for authenticated user	Billing
`/api/rewards`	GET	View earned reward points balance	Billing
`/api/payments/pay`	POST	Submit a new payment transaction	Payment

SECTION 12

Auth & Security Design

CredVault implements a **Layered Security Model** with defense-in-depth principles at every layer of the stack.

1

Transport Security

API Gateway handles SSL termination. Internal service-to-service communication runs on an isolated Docker private network.

2

JWT Authentication

OAuth2-style JWT implementation. Tokens carry Claims — `UserId`, `Email`, `Roles` — and are signed with HMAC-SHA256.

3

Token Rotation Strategy

Access Token: Short-lived (15 min), held in JS memory only. **Refresh Token:** Long-lived (7 days), stored in HTTP-only secure cookie, verified against the database on every rotation.

4

Multi-Factor Authentication

OTP required on every login. 6-digit code generated with a cryptographically secure random generator, stored with 5-minute expiry, consumed and invalidated after a single use.

5

Middleware Guards

Frontend: `AuthInterceptor` auto-attaches JWT to every outgoing HTTP request. **Backend:** `AuthenticationMiddleware` validates every token's signature and claims before any controller receives the request.

SECTION 13

Challenges & Solutions

⚠ CHALLENGE

Inter-service Data Consistency

✓ SOLUTION

Instead of Distributed Transactions (2PC), which introduce blocking latency and tight coupling, the system uses **Eventual Consistency via RabbitMQ**. If Billing is temporarily down when a PaymentCompletedEvent fires, RabbitMQ holds the message. Once Billing recovers, it processes the event and marks the bill paid — no manual intervention, no data loss.

⚠ CHALLENGE

UI Performance with Complex Reactive State

✓ SOLUTION

Implemented a **hybrid state model**: Angular Signals for local, component-level UI state (button spinners, form inputs) and NgRx for global cross-component state (user profile, card list). Fine-grained DOM updates are instant, while shared data stays consistent across every page in the application.

⚠ CHALLENGE

Secure Payment Simulation without a Live Gateway

✓ SOLUTION

Implemented a **Saga-lite pattern** inside the `MakePaymentCommandHandler`. Every step of the multi-service transaction is tracked via state transitions — Processing → Completed or Failed. If a failure occurs mid-flow, a compensation event is published to roll back or alert downstream services atomically.

SECTION 14

Results & Outcomes

⚡

High Throughput

Capable of processing thousands of transactions asynchronously without interdependency bottlenecks between services.

🔒

Bank-Grade Security

MFA, rotating JWTs, BCrypt hashing, and HTTP-only cookies deliver a layered, hardened security posture.

🎨

Premium User Experience

Glassmorphic UI with Three.js 3D card visualizations and ECharts financial dashboards for a genuinely premium feel.

📊

Full Financial Visibility

Automated bill tracking, real-time payment status, and reward point accumulation on a single unified dashboard.

SECTION 15

Learnings & Recommendations

1

Ocelot → Kong / Istio

As the service count grows, migrating to a service mesh like Istio or enterprise gateway Kong would provide better observability, traffic management, and circuit breaking out of the box.

2

Event Sourcing for Payments

Implementing Event Sourcing via EventStoreDB would deliver an immutable audit trail for financial regulators — every state change stored as a fact, never overwritten.

3

BFF Pattern

A Backend for Frontend layer would tailor response shapes for mobile and web separately, reducing gateway round trips and improving perceived performance on constrained networks.

4

Zoneless Angular

Exploring `provideExperimentalZonelessChangeDetection()` in Angular 19 could eliminate `zone.js` overhead entirely, boosting rendering performance further alongside the existing Signals adoption.

CredVault — Technical Case Study

Prepared by **Antigravity** · Senior Software Architect · 2025

This document is confidential and intended for internal review only.